

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Дисциплина: Компьютерные системы и сети

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему

**ВЕБ-ПРИЛОЖЕНИЕ ДЛЯ МОНИТОРИНГА И АНАЛИЗА
ИНФОРМАЦИИ О ПОГОДЕ**

БГУИР КП 6-05-0612-01 014 ПЗ

Студент

Заверач С.П.

Руководитель

Болтак С.В.

Минск 2026

СОДЕРЖАНИЕ

Введение.....	5
1 Анализ предметной области	6
1.1 Сравнительный анализ существующих решений.....	6
1.2 Постановка задачи	9
2 Проектирование программного средства	11
2.1 Архитектура системы и структура сетевого взаимодействия.....	11
2.2 Проектирование взаимодействия с внешним API	12
2.3 Проектирование пользовательского интерфейса.....	14
3 Разработка программного средства.....	16
3.1 Реализация серверной части веб-приложения	16
3.2 Реализация механизмов получения и обработки данных	17
3.3 Реализация системы авторизации и персонализации пользователей....	17
3.4 Реализация клиентской части веб-приложения	20
4 Тестирование программного средства.....	23
5 Руководство пользователя.....	27
Заключение	32
Список используемых источников.....	33
Приложение А (обязательное) Листинг кода с комментариями.....	34
Приложение Б (обязательное) Схема алгоритма работы системы	42

ВВЕДЕНИЕ

В современном мире своевременный доступ к точным метеорологическим данным является важным как для планирования повседневной деятельности человека, так и для функционирования различных отраслей экономики, включая логистику, сельское хозяйство и городское хозяйство. С развитием веб-технологий традиционные методы получения прогнозов уступают место интерактивным аналитическим платформам. Такие системы позволяют отображать текущие показатели, фильтровать и структурировать информацию, обеспечивая пользователю индивидуальную настройку рабочей среды.

Целью данного курсового проекта является проектирование и разработка веб-приложения для мониторинга и структурирования метеорологической информации по городам.

Для достижения поставленной цели необходимо решить следующие задачи:

Провести анализ предметной области, выявить ключевые требования к сервисам визуализации погоды и сформировать функциональные требования к разрабатываемой системе. Спроектировать архитектуру веб-приложения на базе фреймворка Spring Boot. Реализовать программный модуль взаимодействия с удаленным сервером OpenWeatherMap для получения актуальных метеорологических сводок и краткосрочных прогнозов в формате JSON. Создать динамический пользовательский интерфейс с помощью шаблонизатора Thymeleaf и фреймворка Bootstrap, реализовав функции поиска, добавления регионов в список избранного и кастомизации панели быстрого доступа. Проверить работоспособность программного комплекса в ходе функционального тестирования.

Актуальность работы заключается в необходимости создания легковесных, независимых от платформы решений для агрегации данных из открытых API. Использование связки Spring Boot и Thymeleaf позволяет минимизировать затраты на развертывание приложения, а применение локального кэширования в JSON-файлах обеспечивает базовую автономность и высокую скорость отклика интерфейса при повторных запросах пользователя, что делает итоговый продукт конкурентоспособным среди существующих аналогов.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Сравнительный анализ существующих решений

Данный курсовой проект посвящен проектированию и практической реализации многопользовательского веб-приложения, предназначенного для мониторинга, структурирования и анализа метеорологической информации в реальном времени. Основная цель работы заключается в создании сервиса, который позволяет пользователю формировать собственное информационное пространство: управлять списками избранных локаций и настраивать персональные панели быстрого доступа к метеоданным. Разрабатываемая система ориентирована на конечного пользователя и предоставляет инструменты для оперативного анализа климатических параметров в различных точках планеты с сохранением индивидуальных настроек между сессиями.

Разрабатываемое программное средство предоставляет пользователю комплекс функциональных возможностей для эффективного взаимодействия с метеорологической информацией. В систему заложен инструмент поиска текущих погодных параметров по наименованию населенного пункта с использованием клиентского скрипта для динамического подбора названий городов через геокодирование. Приложение позволяет проходить процедуру аутентификации, после чего пользователю открывается доступ к персональному кабинету. В нем реализованы функции формирования списка избранных локаций (отметка «звездочкой»), просмотр подробного пятидневного прогноза и кастомизация верхней панели быстрого доступа, состоящей из пяти приоритетных городов. Дополнительно реализована система ранжирования истории запросов по температуре, стране, алфавитному порядку и погодным условиям, работающая на стороне клиента.

Основу архитектуры разработанной системы составляет веб-приложение на базе фреймворка Spring Boot, реализующее паттерн MVC (Model-View-Controller). Безопасность данных и разграничение доступа обеспечиваются модулем Spring Security с применением криптографического алгоритма BCrypt для хэширования паролей. Серверная логика отвечает за десериализацию данных из внешнего REST API OpenWeatherMap и управление локальным репозиторием на основе JSON-файлов, где для каждого зарегистрированного пользователя выделено независимое пространство для хранения истории и предпочтений. Визуальная часть построена с использованием шаблонизатора Thymeleaf и библиотеки Bootstrap, что обеспечивает адаптивность интерфейса.

Для проведения сравнительного анализа программных решений в области отображения метеоданных были выбраны три популярных веб-ресурса с различной архитектурной концепцией.

Первым рассмотренным аналогом является глобальный портал Gismeteo[1]. Он предоставляет исчерпывающий объем информации, включая геомагнитную обстановку и подробные карты осадков. Однако портал перегружен избыточными аналитическими блоками, коммерческой рекламой и тяжелым медиа-контентом. Это существенно замедляет скорость загрузки страниц, а для сохранения списка избранных городов требуется привязка к сложной экосистеме портала. Пример интерфейса данного портала представлен ниже.

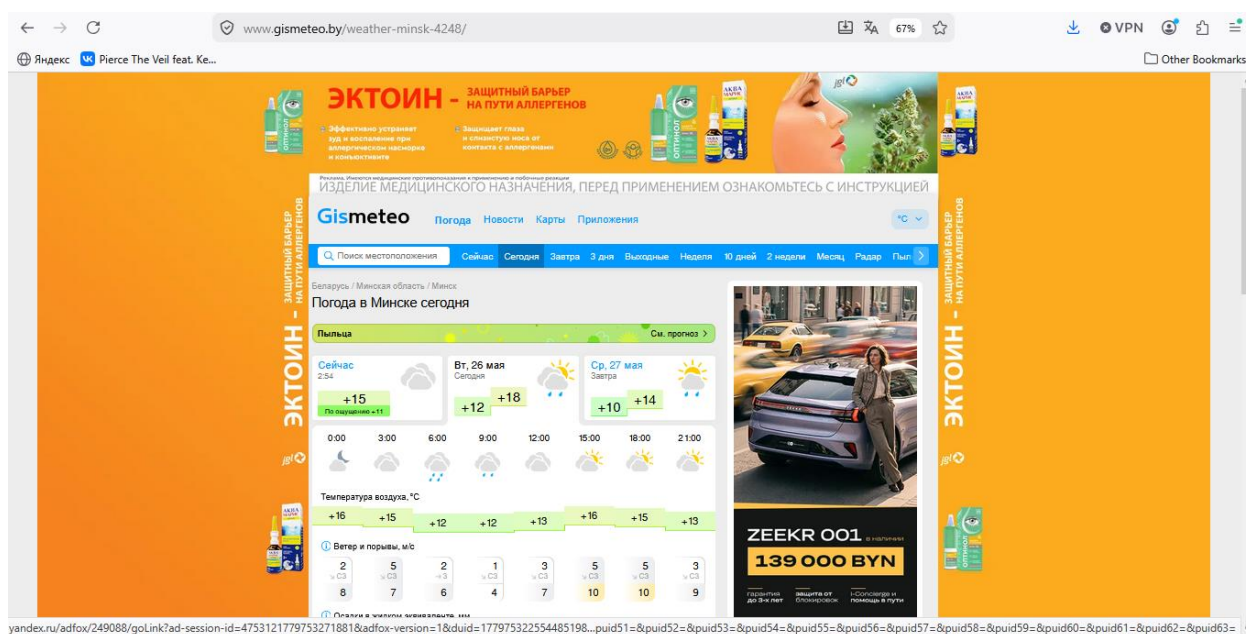


Рисунок 1.1 – Интерфейс платформы Gismeteo

Вторым решением выступает сервис Яндекс.Погода[2]. Платформа использует технологии машинного обучения для сверхточного локального прогнозирования. Интерфейс выполнен в современном стиле и полностью адаптивен. Главным ограничением является закрытость экосистемы: полноценное сохранение настроек жестко привязано к авторизации через единый облачный профиль «Яндекс ID», а бесплатный доступ к их API для сторонней разработки крайне ограничен. Пример визуального отображения данных приведен на следующем изображении.

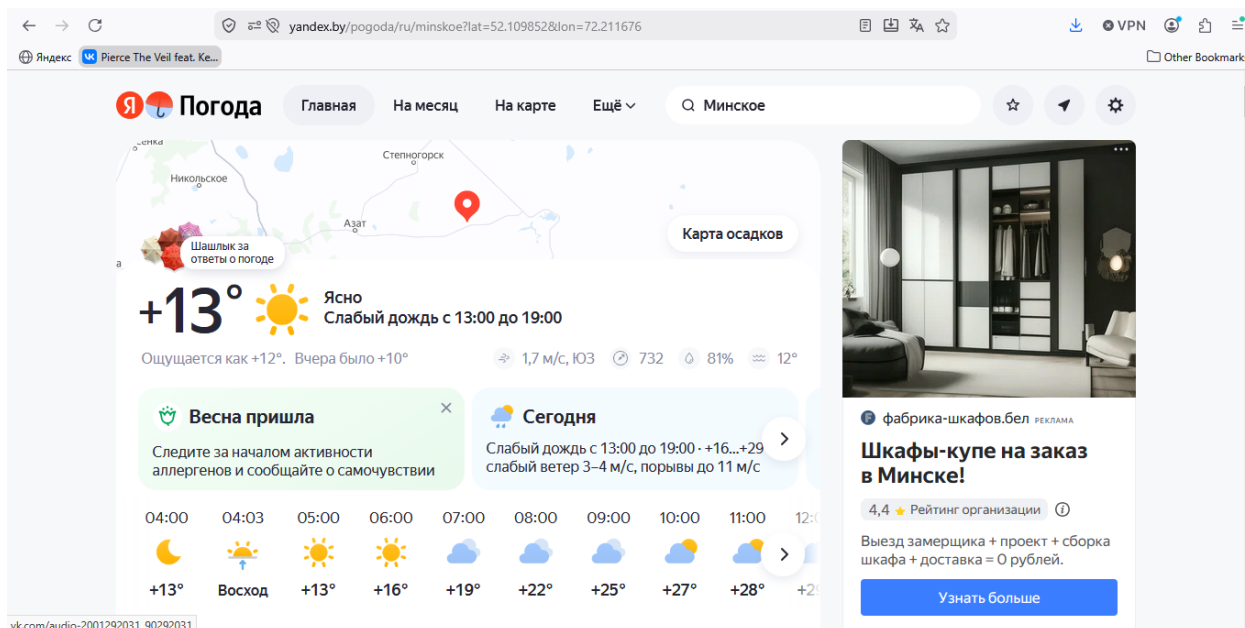


Рисунок 1.2 – Интерфейс платформы Яндекс.Погода

Третьим объектом стал специализированный сервис wttr.in, предоставляющий сводки в виде ASCII-графики. Сервис ориентирован на минимализм и скорость, однако полное отсутствие интерактивного графического интерфейса, невозможность визуального редактирования списков популярных городов через формы и отсутствие инструментов табличной сортировки делают его неудобным для массовой аудитории. Внешний вид данного сервиса продемонстрирован далее.

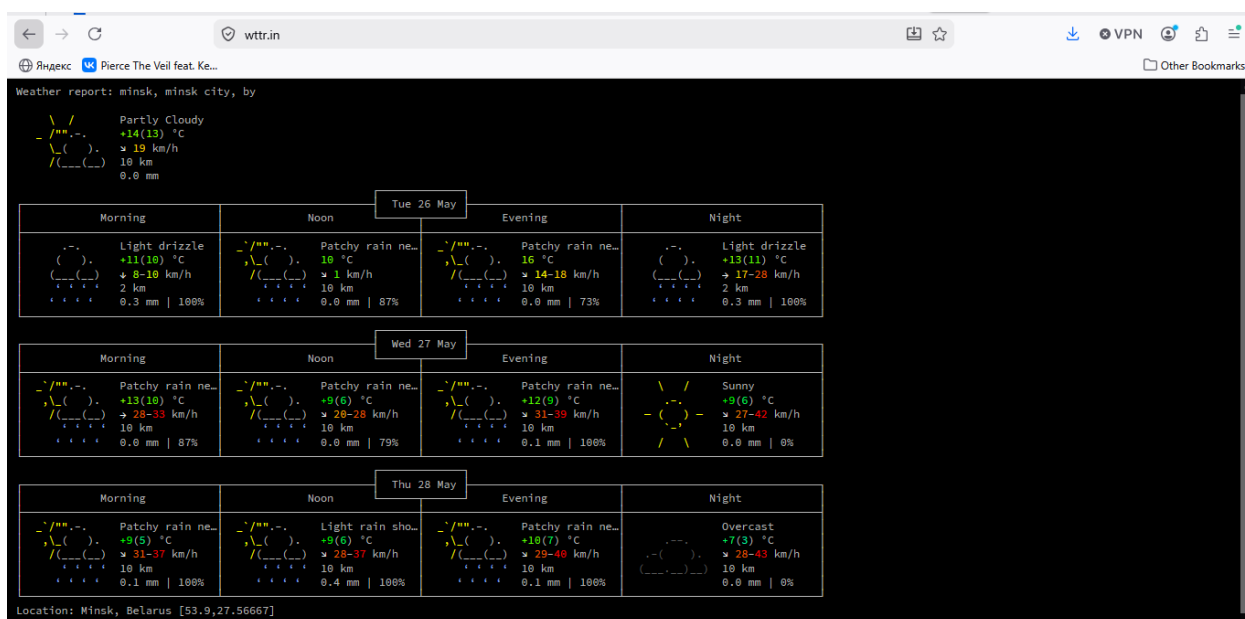


Рисунок 1.3 – Интерфейс платформы wttr.in

В конечном итоге, проведенный сравнительный анализ позволяет обосновать актуальность разработки собственного приложения. Существующие коммерческие порталы избыточны и перегружены рекламой, а консольные утилиты лишены наглядности. Разрабатываемое приложение на Spring Boot должно получиться легковесным с функциональным монитором, который объединяет данные OpenWeatherMap[3] с возможностью персональной кастомизации данных в локальных JSON-файлах, обеспечивая пользователю высокую скорость работы и конфиденциальность настроек.

1.2 Постановка задачи

В рамках данного курсового проекта необходимо спроектировать и реализовать многопользовательское веб-приложение, предназначенное для оперативного мониторинга метеорологической обстановки и получения краткосрочных прогнозов на основе данных внешних метеослужб. Система должна стать персональным инструментом пользователя для структурирования погодных сводок по интересующим его географическим локациям.

Основной целью работы является создание клиент-серверного приложения, обеспечивающего интеграцию с внешним REST[4]-интерфейсом OpenWeatherMap. Через данную службу должны выполняться запросы на получение актуальных физических параметров атмосферы (температура, влажность, скорость и направление ветра, описание облачности) и детализированного прогноза на пятидневный период. Система должна обеспечивать корректную обработку входящих данных в формате JSON, их десериализацию в программные объекты и визуализацию в удобном для пользователя виде.

Разрабатываемое программное средство должно поддерживать систему регистрации и аутентификации пользователей для обеспечения персонализации данных. Каждый зарегистрированный пользователь должен иметь возможность:

- осуществлять поиск текущей погоды в любой точке мира с использованием инструментов автоматического подбора названий городов;
- формировать индивидуальный список избранных локаций для быстрого отслеживания динамики изменений;
- кастомизировать главную панель мониторинга, выбирая до пяти приоритетных городов для отображения в виде информационных карточек;
- просматривать расширенный прогноз погоды на 5 дней с детализацией по датам;

– управлять историей запросов с возможностью мгновенной сортировки данных по температуре, стране и погодным условиям.

Архитектура системы должна быть построена на базе языка программирования Java и фреймворка Spring Boot[5]. Серверная часть приложения обязана обеспечивать безопасную обработку HTTP-запросов, реализацию логики взаимодействия с внешним API и управление локальным хранилищем данных. Особенностью реализации является использование репозитория на основе структурированных JSON-файлов вместо классических реляционных баз данных, что позволяет повысить портативность системы. За безопасность и разграничение прав доступа должен отвечать модуль Spring Security с применением механизмов хэширования паролей.

Клиентская часть приложения должна быть реализована с использованием шаблонизатора Thymeleaf[6]. Для формирования визуального интерфейса и обеспечения его адаптивности необходимо использовать средства CSS (фреймворк Bootstrap) . Интерактивные функции, такие как динамическая сортировка таблиц и асинхронное взаимодействие с API геокодирования, должны быть реализованы на языке JavaScript. Взаимодействие между компонентами системы должно осуществляться по протоколу HTTP, обеспечивая стабильный обмен данными в формате JSON[7] между сервером, клиентом и внешней метеослужбой.

2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

2.1 Архитектура системы и структура сетевого взаимодействия

При проектировании веб-приложения для мониторинга метеорологических данных была выбрана классическая клиент-серверная архитектура. Данный подход позволяет разделить систему на две функционально независимые части: серверную (Backend), отвечающую за бизнес-логику, безопасность и обработку данных, и клиентскую (Frontend), обеспечивающую визуализацию интерфейса и взаимодействие с пользователем.

Серверная часть приложения реализована на языке программирования Java[8] с использованием фреймворка Spring Boot. Она выполняет роль центрального узла системы, осуществляя обработку входящих HTTP-запросов, управление сессиями пользователей через модуль Spring Security[9] и взаимодействие с внешним метеорологическим сервисом. Клиентская часть представляет собой веб-интерфейс, работающий в браузере пользователя, который обеспечивает отображение погодных сводок и выполнение интерактивных сценариев.

Взаимодействие между компонентами системы осуществляется по сети Интернет с использованием протокола HTTP. Обмен данными между сервером приложения, клиентом и внешней метеослужбой выполняется в формате JSON. Общая структура системы включает следующие основные компоненты:

- клиентское веб-приложение на базе Thymeleaf и Bootstrap;
- сервер веб-приложения (Spring Boot);
- локальное хранилище данных пользователей в формате JSON;
- внешняя метеорологическая служба OpenWeatherMap API.

Ключевым аспектом сетевого взаимодействия является интеграция с внешним REST API. При вводе пользователем поискового запроса сервер формирует структурированный HTTP-запрос к API OpenWeatherMap, содержащий уникальный ключ доступа и параметры локации. Полученный от внешней службы ответ в формате JSON подвергается десериализации на стороне сервера в объекты классов моделей (WeatherData, ForecastDay), после чего данные передаются в слой представления для отображения пользователю.

Для хранения информации о пользователях, их учетных записях, индивидуальных настройках популярных городов и истории запросов вместо реляционной базы данных (СУБД) используется файловое хранилище на основе JSON-репозитория. Серверная часть обеспечивает чтение и запись данных с использованием библиотеки Jackson, что гарантирует высокую

портативность приложения и отсутствие зависимости от внешних серверных СУБД. При регистрации или изменении настроек избранных городов сервер обновляет соответствующие JSON-объекты в файловой системе, обеспечивая персистентность данных.

Клиентская часть приложения построена с использованием серверного шаблонизатора Thymeleaf, который динамически формирует HTML-код на основе данных, предоставленных контроллером. Для обеспечения интерактивности без полной перезагрузки страниц используются элементы JavaScript. В частности, реализован механизм асинхронного получения географических подсказок при вводе города и алгоритмы мгновенной сортировки табличных данных на стороне клиента.

Серверная часть построена по модульному принципу и включает в себя:

- слой контроллеров (Controller) для маршрутизации API и веб-запросов;
- сервисный слой (Service) для инкапсуляции логики работы с внешним API;
- слой репозитория (Repository) для управления файловым хранилищем JSON;
- модуль безопасности (Security) для аутентификации и авторизации.

Выбор данной архитектуры обусловлен необходимостью создания легковесного решения, способного эффективно работать с внешними данными в реальном времени. Использование Spring Boot позволяет минимизировать затраты на конфигурацию инфраструктуры, а JSON-ориентированное хранилище обеспечивает быструю развертку приложения в любой среде. Следовательно, выбранная архитектура гарантирует надежное взаимодействие всех компонентов системы и позволяет реализовать персонализированный мониторинг погоды через современный веб-интерфейс.

2.2 Проектирование взаимодействия с внешним API

Проектируемое веб-приложение по своей архитектурной сути является метеорологическим агрегатором, функционирующим по принципу «посредника» между конечным пользователем и профессиональной метеослужбой. Взаимодействие между компонентами системы строится на основе REST-принципов, где серверная часть приложения инициирует запросы к внешнему сервису OpenWeatherMap, обрабатывает полученные массивы данных и транслирует их в слой представления.

Сетевое взаимодействие в системе реализуется по протоколу HTTP. Основным форматом передачи структурированной информации является JSON, который выбран благодаря своей легкости и нативной поддержке в

экосистеме Spring Boot через библиотеку Jackson. Процесс обмена данными инициируется при выполнении пользователем любого активного действия в интерфейсе: поиске города, обновлении главной панели или переходе к детальному прогнозу.

В рамках проектирования были выделены три ключевых вектора взаимодействия с внешним API:

1. Получение оперативных метеоданных. Для заполнения информационных карточек на главной панели и формирования записей в таблице истории запросов сервер обращается к эндпоинту Current Weather Data. В ответ система получает JSON-объект, содержащий текущую температуру, показатели влажности, скорость ветра и идентификатор погодного состояния. На стороне сервера происходит десериализация этого объекта в Java-модель WeatherData, которая затем обогащается данными о статусе «избранного» из локального хранилища пользователя.

2. Формирование долгосрочных прогнозов. При переходе пользователя на страницу детального обзора система задействует API-ресурс 5 Day Forecast. Сложность данного взаимодействия заключается в обработке многоуровневого массива данных, содержащего замеры с трехчасовым шагом. Логика приложения проектируется таким образом, чтобы фильтровать этот массив и извлекать только релевантные значения для отображения дневного прогноза на пятидневный период.

3. Геокодирование и интерактивный поиск. Для обеспечения работы функции автоматического подбора названий городов (Autocomplete) спроектировано взаимодействие клиентской части приложения с API Geocoding. Это позволяет выполнять асинхронные запросы непосредственно из браузера пользователя при вводе текста в поисковую строку. Система возвращает список подходящих географических наименований с указанием стран, что минимизирует риск ввода некорректных данных.

Безопасность сетевого взаимодействия обеспечивается использованием уникальных API-ключей, которые инкапсулированы в конфигурационных файлах сервера и передаются внешнему сервису в качестве обязательного параметра. Кроме того, в системе предусмотрены механизмы обработки сетевых исключений: в случае недоступности внешнего API или превышения лимитов запросов, приложение проектируется таким образом, чтобы корректно отображать данные из локального кэша или выводить информативное сообщение пользователю.

Такой подход к проектированию взаимодействия позволяет системе оставаться легковесной, так как основная нагрузка по сбору и анализу первичных метеоданных ложится на внешнюю профессиональную службу, в то время как локальное приложение концентрируется на персонализации,

хранении предпочтений пользователей и обеспечении удобного пользовательского интерфейса.

2.3 Проектирование пользовательского интерфейса

Функциональные возможности разрабатываемого веб-приложения были определены с учётом специфики современных метеорологических сервисов и требований к системам персонализации данных. Основной задачей проектирования являлось создание интерфейса, обеспечивающего интуитивно понятное взаимодействие пользователя с динамически меняющейся погодной информацией.

В приложении предусмотрено разделение пользователей на две категории в зависимости от уровня доступа к функциям системы: неавторизованный пользователь (гость) и зарегистрированный пользователь. Неавторизованный пользователь имеет доступ к базовым функциям системы: поиску текущей погоды в любом населенном пункте и просмотру пятидневного прогноза. При этом его история поисковых запросов и настройки интерфейса не сохраняются между сессиями.

Зарегистрированный пользователь получает полный спектр возможностей персонализации. После прохождения процедуры аутентификации пользователю становятся доступны следующие функции:

- формирование индивидуального списка избранных локаций (отметка «звездочкой»);
- кастомизация главной панели мониторинга путем выбора пяти приоритетных городов для мгновенного отображения;
- сохранение и управление личной историей поисковых запросов;
- доступ к расширенному анализу метеоданных через сортируемую таблицу запросов.

Одной из ключевых функций системы является поиск городов. При проектировании данной возможности учитывались требования к минимизации ошибок ввода. Реализован механизм асинхронного взаимодействия с API геокодирования, который предоставляет пользователю список подходящих названий городов непосредственно в процессе набора поискового запроса. Это позволяет динамически подбирать корректные географические идентификаторы без перезагрузки страницы.

Важной функциональной составляющей является модуль визуализации прогнозов. Система обеспечивает получение и обработку данных о состоянии атмосферы на пятидневный период. Проектирование интерфейса данного модуля велось с упором на наглядность: для каждого временного интервала

предусмотрено отображение иконки погодных условий, температурного режима и текстового описания состояния облачности.

Для организации удобной работы с накопленными данными в системе спроектирован механизм интерактивной сортировки. Пользователь может мгновенно ранжировать записи в таблице истории по следующим критериям:

- наименование населенного пункта;
- страна расположения;
- текущая температура (в порядке возрастания и убывания);
- состояние погодных условий.

При проектировании интерфейса особое внимание уделялось адаптивности и скорости отклика. Клиентская часть реализована с использованием фреймворка Bootstrap, что гарантирует корректное отображение системы на различных типах устройств (ПК, планшеты, смартфоны). Для взаимодействия клиентской части с сервером используются асинхронные запросы и элементы JavaScript, что позволяет обновлять результаты поиска и сортировать данные в таблицах без полной перезагрузки страницы. Такой подход обеспечивает плавную навигацию и повышает удобство использования сервиса.

В конечном итоге, спроектированные функциональные возможности и интерфейсные решения обеспечивают реализацию полноценного метеорологического веб-приложения с поддержкой персонализации, глубокого анализа данных и удобного взаимодействия пользователя с внешней метеослужбой через современный графический интерфейс.

3 РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

3.1 Реализация серверной части веб-приложения

Основным компонентом серверной части является веб-приложение, реализованное на языке программирования Java с использованием фреймворка Spring Boot. Точкой входа приложения является класс WeatherApplication, содержащий метод main. При запуске выполняется автоматическая конфигурация контекста Spring, инициализация встроенного HTTP-сервера Apache Tomcat и регистрация всех программных компонентов (бинов) в контейнере управления.

Центральным узлом обработки клиентских запросов является контроллер WeatherController. Маршрутизация запросов реализована с помощью аннотаций @GetMapping и @PostMapping, которые связывают URL-адреса и HTTP-методы с соответствующими методами-обработчиками. В системе реализована поддержка методов GET (для получения страниц и данных) и POST (для выполнения операций изменения состояния системы, таких как поиск, авторизация или сохранение настроек).

Для обработки метеорологических данных и взаимодействия с внешним API реализован сервисный слой в лице класса WeatherApiService. В процессе работы сервис использует объект RestTemplate для выполнения асинхронных HTTP-запросов к серверу OpenWeatherMap. Полученные ответы десериализуются из формата JSON в Java-объекты моделей WeatherData и ForecastDay с помощью библиотеки Jackson. Для обеспечения гибкости конфигурации API-ключи и настройки внешних сервисов вынесены в файл параметров application.properties.

В системе реализована группа основных обработчиков в составе контроллера:

- index — формирование главной страницы мониторинга, включающее сбор данных о популярных городах и истории запросов;
- fetch — инициация поиска метеоданных для нового населенного пункта и сохранение результата в репозиторий;
- forecast — получение и визуализация детализированного пятидневного прогноза для выбранного города;
- toggleFavorite — изменение статуса «избранного» для конкретной локации в профиле пользователя;
- updatePopular — обновление настроек главной панели быстрого доступа.

Для обеспечения многопользовательского доступа и персонализации данных реализован модуль безопасности на базе Spring Security.

Конфигурация безопасности вынесена в класс `SecurityConfig`, где определены правила доступа к ресурсам: страницы аутентификации и регистрации открыты для всех пользователей, в то время как доступ к основному функционалу мониторинга требует обязательной авторизации.

Процесс регистрации пользователей реализован в методе `registerUser`. Система выполняет проверку на уникальность имени пользователя через `UserRepository`, после чего пароль подвергается одностороннему хэшированию с использованием алгоритма `BCrypt` через компонент `PasswordEncoder`. Хранение данных осуществляется в локальном репозитории на основе структурированного JSON-файла `users.json`. Работа с файловым хранилищем инкапсулирована в классе `UserRepository`, который с помощью `ObjectMapper` обеспечивает чтение и запись массивов данных, гарантируя персистентность настроек и истории каждого пользователя.

Механизм пользовательских сессий управляется `Spring Security` автоматически после успешной аутентификации. Текущий авторизованный пользователь определяется в методах контроллера через объект `Principal`, что позволяет приложению изолировать данные разных аккаунтов. Взаимодействие между серверной логикой и клиентским интерфейсом осуществляется посредством передачи объектов-моделей в шаблонизатор `Thymeleaf`, который генерирует динамический HTML-код на основе текущего состояния данных.

Таким образом, реализованная серверная часть на базе `Spring Boot` обеспечивает эффективную обработку HTTP-запросов, интеграцию с внешним REST API, управление пользовательскими сессиями и надежное хранение персональных данных в формате JSON.

3.2 Реализация механизмов получения и обработки данных

Подсистема получения и обработки метеорологических данных обеспечивает взаимодействие с внешними информационными ресурсами, десериализацию входящих потоков данных и их сохранение в локальном хранилище в привязке к конкретным профилям пользователей.

Получение актуальных данных выполняется сервисным слоем в лице метода `getRemoteWeather` класса `WeatherApiService`. Для обмена данными используется сетевое взаимодействие по протоколу HTTP с внешним REST-интерфейсом. Процесс получения данных инициируется при поиске города пользователем или при обновлении главной панели мониторинга.

После получения ответа от внешней службы в формате JSON, сервер выполняет проверку корректности данных и их десериализацию в Java-

объекты модели WeatherData. Процесс обработки данных включает несколько критических этапов:

- проверка статус-кода HTTP-ответа (обработка ситуаций с некорректным наименованием населенного пункта или отсутствием связи);
- разбор структуры JSON-ответа с использованием библиотеки Jackson;
- приведение физических величин к выбранной системе измерения (метрическая система, градусы Цельсия);
- локализация текстовых описаний погодных условий на русский язык.

В приложении реализован механизм предотвращения дублирования данных в истории запросов. При сохранении нового результата поиска в методе `save` класса `UserRepository` выполняется проверка текущего списка истории пользователя. Если запись для данного населенного пункта уже существует (сравнение выполняется по наименованию города без учета регистра), старая запись удаляется, и на её место записывается актуальный объект с обновленными параметрами температуры и времени замера. Такой подход позволяет поддерживать историю запросов в актуальном состоянии и оптимизировать объем хранимых данных.

Для обеспечения корректной навигации и работы с URL-адресами в системе реализована функция кодирования имен городов с использованием `UriUtils.encode` в кодировке UTF-8. Это позволяет безопасно передавать кириллические названия населенных пунктов в параметрах запросов и предотвращать ошибки при формировании ссылок на детализированные прогнозы.

Хранение обработанной метеорологической информации осуществляется в рамках объекта пользователя `User`. После выполнения операций поиска или обновления статуса «избранного», модифицированный объект пользователя передается в репозиторий для сериализации в JSON-файл `users.json`. Для каждого города сохраняются следующие параметры:

- наименование населенного пункта и код страны;
- текущая температура, влажность и скорость ветра;
- текстовое описание погодных условий и идентификатор графической иконки;
- дата и время получения данных;
- признак нахождения города в списке избранного.

При изменении настроек панели быстрого доступа выполняется обновление соответствующего массива данных в объекте пользователя с последующей перезаписью файла. Для обеспечения безопасности данных при операциях записи используется метод `writerWithDefaultPrettyPrinter`, который формирует структурированный и читаемый JSON-файл, что упрощает отладку и мониторинг состояния системы.

В результате проделанной работы, реализованная подсистема получения и обработки метеорологических данных обеспечивает надежное взаимодействие с внешним API, управление историей поисковых запросов и гарантирует сохранность персональных настроек и предпочтений пользователей в файловом хранилище системы.

3.3 Реализация системы авторизации и персонализации пользователей

Система авторизации и персонализации данных реализована на основе фреймворка Spring Security, который обеспечивает разграничение прав доступа и защиту пользовательской информации. Основной задачей подсистемы является создание изолированной среды для каждого пользователя, в которой сохраняются его поисковые предпочтения и настройки интерфейса.

Процесс аутентификации пользователей управляется классом конфигурации SecurityConfig. Для обеспечения безопасности в системе реализованы следующие механизмы:

- определение общедоступных маршрутов (/login, /register, ресурсы статики) и закрытых зон, требующих обязательной авторизации;
- использование пользовательской реализации интерфейса UserDetailsService, которая осуществляет поиск учетных данных в локальном JSON-репозитории;
- применение криптографического алгоритма BCrypt для хэширования паролей[10] перед их сохранением, что гарантирует защиту данных даже в случае прямого доступа к файлу хранилища.

Механизм персонализации данных базируется на работе с объектом класса User, который инкапсулирует в себе не только учетные данные, но и персональные массивы информации: список избранных городов и перечень популярных локаций для главной панели. При каждой авторизации сервер идентифицирует текущего пользователя через объект Principal, что позволяет подгружать именно его настройки из общего файла users.json.

Одной из ключевых функций персонализации является управление списком «избранных» городов. Реализованный в WeatherController метод toggleFavorite обеспечивает:

- поиск города в истории запросов конкретного пользователя;
- инверсию логического флага favorite;
- автоматическое добавление города в историю, если он ранее отсутствовал, но был отмечен как избранный;

– сохранение обновленного состояния объекта пользователя в файловой системе.

Дополнительно в приложении реализована возможность кастомизации панели быстрого доступа, состоящей из пяти приоритетных городов. Через метод `updatePopular` пользователь передает список интересующих его локаций, который проходит валидацию на стороне сервера (фильтрация пустых значений и ограничение количества элементов). После обработки новые настройки записываются в профиль пользователя, что позволяет динамически формировать верхний блок информационных карточек при каждом входе в систему.

Персистентное хранение персонализированных данных обеспечивается слоем репозитория `UserRepository`. В процессе работы используется библиотека `Jackson`[15] для сериализации объектов Java в формат JSON. Сохранение данных выполняется атомарно для каждого пользователя: при внесении изменений (добавление города в историю, смена «звездочки») сервер считывает текущий массив пользователей, модифицирует данные активного аккаунта и производит полную перезапись файла с использованием форматирования `PrettyPrinter`, обеспечивая целостность и читаемость структуры данных.

На клиентской стороне персонализация проявляется в динамическом формировании интерфейса. Благодаря интеграции `Spring Security` с шаблонизатором `Thymeleaf`, система автоматически отображает имя текущего пользователя и применяет соответствующие CSS-классы[11] для визуального выделения «избранных» городов (желтая иконка звезды) в общей таблице истории.

Таким образом, реализованная система авторизации и персонализации на базе `Spring Security` и JSON-хранилища обеспечивает надежную защиту пользовательских аккаунтов и позволяет каждому пользователю формировать индивидуальное информационное пространство для мониторинга метеорологических данных.

3.4 Реализация клиентской части веб-приложения

Клиентская часть приложения представляет собой современный адаптивный веб-интерфейс, реализованный на стыке серверного шаблонизирования `Thymeleaf` и клиентского программирования на языке JavaScript. Основной упор при разработке был сделан на обеспечение высокой скорости отклика интерфейса и наглядность представления метеорологических данных. Архитектура фронтенда сочетает в себе

надежность серверного рендеринга и динамизм интерактивных элементов, что позволяет создать комфортную среду для пользователя.

Визуальное оформление системы базируется на использовании фреймворка Bootstrap 5[12], который позволил реализовать модульную структуру страниц. Основной файл разметки index.html проектировался с использованием контейнерной верстки и системы сеток (Grid System), что обеспечивает логическое разделение интерфейса на несколько приоритетных областей:

1. Header (Верхняя панель): содержит название сервиса и блок управления учетной записью. Благодаря интеграции со Spring Security, здесь динамически отображается имя текущего пользователя, а также реализована форма безопасного завершения сессии (Logout).

2. Dashboard (Панель популярных городов): реализована в виде набора из пяти информационных карточек (card). Каждая карточка содержит не только текстовую информацию, но и графическую иконку погодных условий, получаемую по прямым ссылкам с серверов OpenWeatherMap, а также температурный индикатор, выделенный жирным начертанием для быстрого считывания.

3. Search & Filter Zone: включает в себя поисковую строку с градиентным фоном и блок кнопок для управления сортировкой табличных данных.

4. History Table (Таблица запросов): является наиболее сложным элементом интерфейса. В ней реализовано условное форматирование: строки с «избранными» городами подсвечиваются иконками заполненных звезд, а вновь найденные города выделяются специальным анимационным эффектом.

Для обеспечения интерактивности без полной перезагрузки страниц в приложении реализована глубокая логика на языке JavaScript[13]. Ключевой функцией является система асинхронного поиска городов (Autocomplete). Скрипт через слушатель событий addEventListener отслеживает каждое нажатие клавиши в поисковом поле. При вводе более двух символов инициируется Fetch API запрос к внешнему геокодирующему сервису. Полученные данные в формате JSON мгновенно преобразуются в элементы списка option внутри тега datalist, что позволяет пользователю выбрать корректное название города из выпадающего списка, сводя риск опечаток к минимуму.

Особое внимание в реализации клиентской части уделено механизму мгновенной сортировки данных. В отличие от традиционных решений, требующих обращения к серверу, функция sortTable выполняет манипуляции непосредственно с DOM-деревом браузера. Алгоритм сортировки учитывает тип данных:

- для текстовых полей (Город, Страна) применяется лексикографическое сравнение;
- для температурных значений используется извлечение численных данных из специальных атрибутов data-order, что позволяет корректно ранжировать положительные и отрицательные температуры;
- отдельным приоритетом в сортировке обладают записи, помеченные как «избранные», которые всегда остаются в верхней части списка для удобства пользователя.

Для настройки персонализированных параметров в системе реализован модуль модальных окон. Модальное окно settingsModal позволяет пользователю в интерактивном режиме изменять перечень приоритетных городов на главной панели. Обработка форм внутри модальных окон выполняется с использованием валидации на стороне клиента, предотвращая отправку пустых или некорректных запросов.

Адаптивность интерфейса реализована с помощью медиа-запросов (media queries) и встроенных классов Bootstrap. При переходе на мобильные устройства с малой шириной экрана (менее 768 пикселей) система автоматически скрывает второстепенные колонки таблицы (влажность, ветер, дата), оставляя только критически важную информацию о температуре и погодных условиях. Это позволяет сохранить высокую читаемость интерфейса на любых типах устройств — от персональных компьютеров до смартфонов.

Визуальный отклик системы дополнен CSS-анимациями. В частности, реализован эффект highlight-fade: при добавлении нового города в историю, соответствующая строка таблицы на несколько секунд окрашивается в мягкий желтый цвет и слегка увеличивается в масштабе, плавно возвращаясь к исходному состоянию. Это помогает пользователю мгновенно сориентироваться в обновленном списке данных.

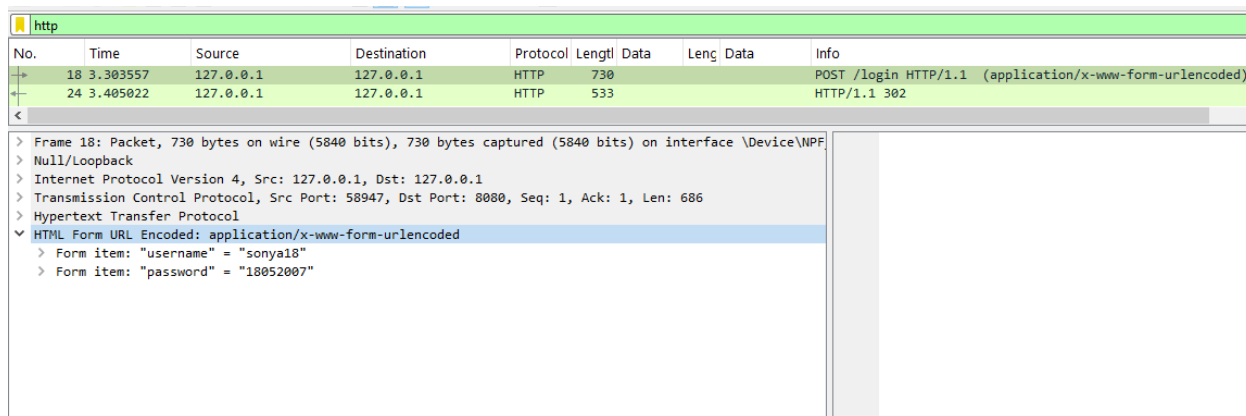
Таким образом, реализованная клиентская часть веб-приложения представляет собой функционально насыщенный, эстетически выверенный и технически оптимизированный интерфейс, который обеспечивает эффективное взаимодействие пользователя с метеорологическим сервисом и предоставляет широкие возможности для анализа и персонализации данных.

4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

Тестирование разработанного веб-приложения выполнялось с целью проверки корректности работы клиентской и серверной частей системы, механизмов сетевого взаимодействия и системы разграничения прав доступа. В процессе разработки использовалось ручное функциональное тестирование, дополненное анализом сетевого трафика.

Для проверки сетевого взаимодействия и подтверждения корректности работы протокола HTTP[14] использовалась программа Wireshark. Анализ трафика выполнялся в режиме реального времени при прохождении пользователем ключевых сценариев работы с приложением.

Сценарий 1. Аутентификация пользователя. При входе в систему браузер отправляет POST-запрос на эндпоинт /login. На рисунке 4.1 представлен перехваченный HTTP-пакет данного запроса. В разделе полезной нагрузки (HTML Form URL Encoded) видны передаваемые параметры username и password. Успешная аутентификация подтверждается последующим ответом сервера со статусом 302 Found и установкой идентификатора сессии.



The screenshot shows the Wireshark network protocol analyzer interface. The top section displays a list of captured packets. Packet 18 is selected, showing it is an HTTP POST request to /login. The packet details pane on the right shows the structure of the packet, including the Ethernet II, Internet Protocol Version 4, Transmission Control Protocol, and Hypertext Transfer Protocol layers. The Hypertext Transfer Protocol section is expanded, showing the request body as 'HTML Form URL Encoded: application/x-www-form-urlencoded'. Below this, the form items are listed: 'Form item: "username" = "sonya18"' and 'Form item: "password" = "18052007"'. The packet bytes pane at the bottom shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Data	Length	Data	Info
18	3.303557	127.0.0.1	127.0.0.1	HTTP	730				POST /login HTTP/1.1 (application/x-www-form-urlencoded)
24	3.405022	127.0.0.1	127.0.0.1	HTTP	533				HTTP/1.1 302

> Frame 18: Packet, 730 bytes on wire (5840 bits), 730 bytes captured (5840 bits) on interface \Device\NPF...
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 58947, Dst Port: 8080, Seq: 1, Ack: 1, Len: 686
> Hypertext Transfer Protocol
> HTML Form URL Encoded: application/x-www-form-urlencoded
> Form item: "username" = "sonya18"
> Form item: "password" = "18052007"

Рисунок 4.1 – HTTP-пакет с данными аутентификации пользователя

Сценарий 2. Запрос метеоданных (Поиск города). При вводе названия населенного пункта в поисковую строку и нажатии кнопки «Найти», клиентская часть инициирует POST-запрос на адрес /fetch. На рисунке 4.2 зафиксирован момент передачи названия города («Кобрин») в теле HTTP-запроса. Анализ пакета подтверждает использование кодировки UTF-8 для корректной передачи кириллических символов и наличие сессионных куки, идентифицирующих пользователя.

181	9.450350	127.0.0.1	127.0.0.1	HTTP	732	POST /fetch HTTP/1.1 (application/x-www-form-urlencoded)
187	9.553085	127.0.0.1	127.0.0.1	HTTP	528	HTTP/1.1 302

> Frame 181: Packet, 732 bytes on wire (5856 bits), 732 bytes captured (5856 bits) on interface \Device\NPF > Null/Loopback > Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 > Transmission Control Protocol, Src Port: 58947, Dst Port: 8080, Seq: 1233, Ack: 34104, Len: 688 > Hypertext Transfer Protocol HTML Form URL Encoded: application/x-www-form-urlencoded - Form item: "city" = "Кобрин" - Key: city - Value: Кобрин	<pre> 0020 fd df 06 09 50 18 27 74 f0 dc 00 00 50 4f 53 54 0030 20 2f 66 65 74 63 68 20 48 54 54 50 2f 31 2e 31 0040 0d 0a 48 6f 73 74 3a 20 6c 6f 63 61 6c 68 6f 73 0050 74 3a 38 30 38 30 0d 0a 55 73 65 72 2d 41 67 65 0060 6e 74 3a 20 4d 6f 7a 69 6c 6c 61 2f 35 2e 30 20 0070 28 57 69 6e 64 6f 77 73 20 4e 54 20 31 30 2e 30 0080 3b 20 57 69 6e 36 34 3b 20 78 36 34 3b 20 72 76 0090 3a 31 35 30 2e 30 29 20 47 65 63 6b 6f 2f 32 30 00a0 31 30 30 31 30 31 20 46 69 72 65 66 6f 78 2f 31 00b0 35 30 2e 30 0d 0a 41 63 63 65 70 74 3a 20 74 65 00c0 78 74 2f 68 74 6d 6c 2c 61 70 70 6c 69 63 61 74 00d0 69 6f 6e 2f 78 68 74 6d 6c 2b 78 6d 6c 2c 61 70 00e0 70 6c 69 63 61 74 69 6f 6e 2f 78 6d 6c 3b 71 3d 00f0 30 2e 39 2c 2a 2f 2a 3b 71 3d 30 2e 38 0d 0a 41 0100 63 63 65 70 74 2d 4c 61 6e 67 75 61 67 65 3a 20 0110 65 6e 2d 55 53 2c 65 6e 3b 71 3d 30 2e 39 0d 0a </pre>
--	--

Рисунок 4.2 – Передача поискового запроса в теле HTTP-пакета

Сценарий 3. Получение детализированного прогноза. При переходе к расширенному анализу погоды выполняется GET-запрос по маршруту /forecast/{city}. На рисунке 4.3 представлен ответ сервера на данный запрос (пакет №338). Пакет содержит статус-код 200 ОК и тело ответа в формате HTML[17], сформированное шаблонизатором Thymeleaf. В дампе данных видна структура страницы с подключенными стилями Bootstrap и заголовком прогноза..

330	28.095325	127.0.0.1	127.0.0.1	HTTP	669	GET /forecast/%D0%9C%D0%B8%D0%BD%D1%81%D0%BA HTTP/1.1
338	28.210055	127.0.0.1	127.0.0.1	HTTP	49	HTTP/1.1 200 (text/html)

> Frame 338: Packet, 49 bytes on wire (392 bits), 49 bytes captured (392 bits) on interface \Device\NPF > Null/Loopback > Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 > Transmission Control Protocol, Src Port: 8080, Dst Port: 58947, Seq: 74719, Ack: 3132, Len: 5 > [2 Reassembled TCP Segments (5436 bytes): #336(5431), #338(5)] > Hypertext Transfer Protocol, has 2 chunks (including last chunk) Line-based text data: text/html (148 lines) <pre> <!DOCTYPE html>\r\n <html>\r\n <head>\r\n <meta charset="UTF-8">\r\n <title>Прогноз - Минск</title>\r\n <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.mi <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.0/font/bootstrap </head>\r\n :root {\r\n --main-blue: #004aad;\r\n --light-grey: #f2f4f7;\r\n }\r\n \r\n .forecast-card {\r\n </pre>	<pre> 0000 02 00 00 00 45 00 00 2d 67 1f 40 00 80 06 06 0010 7f 00 00 01 7f 00 00 01 1f 90 e6 43 fd df a4 0020 54 18 8d ee 50 18 27 f0 bb 4f 00 00 30 0d 0e 0030 0a </pre>
---	---

Рисунок 4.3 – Ответ сервера со структурой страницы прогноза

Сценарий 4. Проверка персистентности сессии. Для подтверждения того, что система распознает пользователя при выполнении защищенных действий, был проанализирован заголовок исходящего запроса. На рисунке 4.4 виден HTTP-заголовок Cookie, содержащий актуальный токен JSESSIONID. Это гарантирует, что все вносимые изменения (избранное, история) будут записаны именно в профиль текущего пользователя в файле users.json.

> Frame 330: Packet, 669 bytes on wire (5352 bits), 669 bytes captured (5352 bits) on interface \Device\NPF{...} > Null/Loopback > Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 > Transmission Control Protocol, Src Port: 58947, Dst Port: 8080, Seq: 2507, Ack: 69288, Len: 625 > Hypertext Transfer Protocol		0040 25 44 30 25 42 38 25 44 30 25 42 44 25 44 31 25 80%88%0 0%80%01% 0050 38 31 25 44 30 25 42 41 20 48 54 54 50 2f 31 2e 81%00%8A HTTP/1.1 0060 31 0d 0a 48 6f 73 74 3a 20 6c 6f 63 61 6c 68 6f 1..Host: localho 0070 73 74 3a 38 30 38 30 0d 0a 55 73 65 72 2d 41 67 st:8080..User-Ag 0080 65 6e 74 3a 20 4d 6f 7a 69 6c 6c 61 2f 35 2e 30 ent: Moz illa/5.0 0090 20 28 57 69 6e 64 6f 77 73 20 4e 54 20 31 30 2e (Window s NT 10. 00a0 30 3b 20 57 69 6e 36 34 3b 20 78 36 34 3b 20 72 0; Win64 ; x64; r 00b0 76 3a 31 35 30 2e 30 29 20 47 65 63 6b 6f 2f 32 v:150.0) Gecko/2 00c0 30 31 30 30 31 30 31 20 46 69 72 65 66 6f 78 2f 0100101 Firefox/ 00d0 31 35 30 2e 30 0d 0a 41 63 65 67 70 74 3a 20 74 150.0..A ccept: t 00e0 65 78 74 2f 68 74 6d 6c 2c 61 70 70 6c 69 63 61 ext/html , applica 00f0 74 69 6f 6e 2f 78 68 74 6d 6c 2b 78 6d 6c 2c 61 tion/xht ml+xml,a 0100 70 70 6c 69 63 61 74 69 6f 6e 2f 78 6d 6c 3b 71 pplicati on/xml;q 0110 3d 30 2e 39 2c 2a 2f 2a 3b 71 3d 30 2e 38 0d 0a =0.9,*/* ;q=0.8.. 0120 41 63 63 65 70 74 2d 4c 61 6e 67 75 61 67 65 3a Accept-L anguage: 0130 20 65 6e 2d 55 53 2c 65 6e 3b 71 3d 30 2e 39 0d en-US,en;q=0.9.. 0140 0a 41 63 63 65 70 74 2d 45 6e 63 6f 64 69 6e 67 .Accept- Encoding 0150 3a 20 67 7a 69 70 2c 20 64 65 66 6c 61 74 65 2c : gzip, deflate, 0160 20 62 72 2c 20 7a 73 74 64 0d 0a 43 6f 6e 6e 65 br, zst d..Conne 0170 63 74 69 6f 6e 3a 20 6b 65 65 70 2d 61 6c 69 76 ction: k eep-aliv 0180 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 74 70 e..Refer er: http 0190 3a 2f 2f 6c 6f 63 61 6c 68 6f 73 74 3a 38 30 38 ://local host:808 01a0 30 2f 3f 6e 65 77 43 69 74 79 3d 25 44 30 25 39 0/?newC1 ty=80889 01b0 41 25 44 30 25 42 45 25 44 30 25 42 31 25 44 31 A%00%8E% D0%81%01 01c0 25 38 30 25 44 30 25 42 38 25 44 30 25 42 44 0d %80%00%8 8%00%80.. 01d0 0a 43 6f 6f 6b 69 65 3a 20 4a 63 45 53 53 49 4f .Cookie: JSESSIO 01e0 4e 49 44 3d 42 41 42 34 43 35 34 42 36 39 46 42 NID=BA64 C54869F8 01f0 36 41 39 45 42 37 30 46 32 45 32 38 39 46 36 36 6A9E870F 2E289F66 0200 46 42 37 44 0d 0a 55 70 67 72 61 64 65 2d 49 6e F87D..Up grade-In 0210 73 65 63 75 72 65 2d 52 65 71 75 65 73 74 73 3a secure-R equests: 0220 20 31 0d 0a 53 65 63 2d 46 65 74 63 68 2d 44 65 1..Sec- Fetch-De	
--	--	---	--

Рисунок 4.4 – Передача сессионного идентификатора JSESSIONID

Дополнительно было проведено функциональное тестирование пользовательского интерфейса. Результаты проверки основных функций системы сведены в таблицу 4.1.

Таблица 4.1 – Результаты функционального тестирования

Проверяемая функция	Ожидаемый результат	Фактический результат	Статус
Регистрация пользователя	Создание записи в users.json, переход на логин	Соответствует	Пройден
Аутентификация	Вход выполнен, отображается имя пользователя	Соответствует	Пройден
Поиск погоды (fetch)	Данные получены из API и добавлены в историю	Соответствует	Пройден
Добавление избранное	Статус сохраняется в профиле (желтая звезда)	Соответствует	Пройден
Настройка панели карточек	5 популярных городов обновляются через форму	Соответствует	Пройден
Сортировка таблицы	Мгновенное ранжирование данных по температуре	Соответствует	Пройден

Продолжение таблицы 4.1 - Результаты функционального тестирования

Проверяемая функция	Ожидаемый результат	Фактический результат	Статус
Прогноз на 5 дней	Отображение расширенной сводки по дням	Соответствует	Пройден
Защита ресурсов	Доступ к истории без авторизации невозможен	Соответствует	Пройден
Выход из аккаунта	Сессия аннулируется, переход на страницу входа	Соответствует	Пройден
Обработка некорректного ввода	Поиск несуществующего о города не приводит к сбою системы, ошибка обрабатывается корректно	Соответствует	Пройден
Адаптивность интерфейса	Элементы страницы корректно перестраиваются под размер экрана	Соответствует	Пройден

5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Для запуска веб-приложения необходимо предварительно инициализировать серверную часть системы на базе Spring Boot. После успешного старта сервера пользователь открывает клиентскую часть приложения в любом современном веб-браузере по адресу, указанному в конфигурации (по умолчанию — `http://localhost:8080`).

После открытия приложения пользователь попадает на страницу авторизации. Доступ к основным функциям мониторинга, персонализированным спискам и истории запросов предоставляется только после прохождения процедуры аутентификации.

На рисунке 5.1 представлено главное окно веб-приложения (после авторизации).

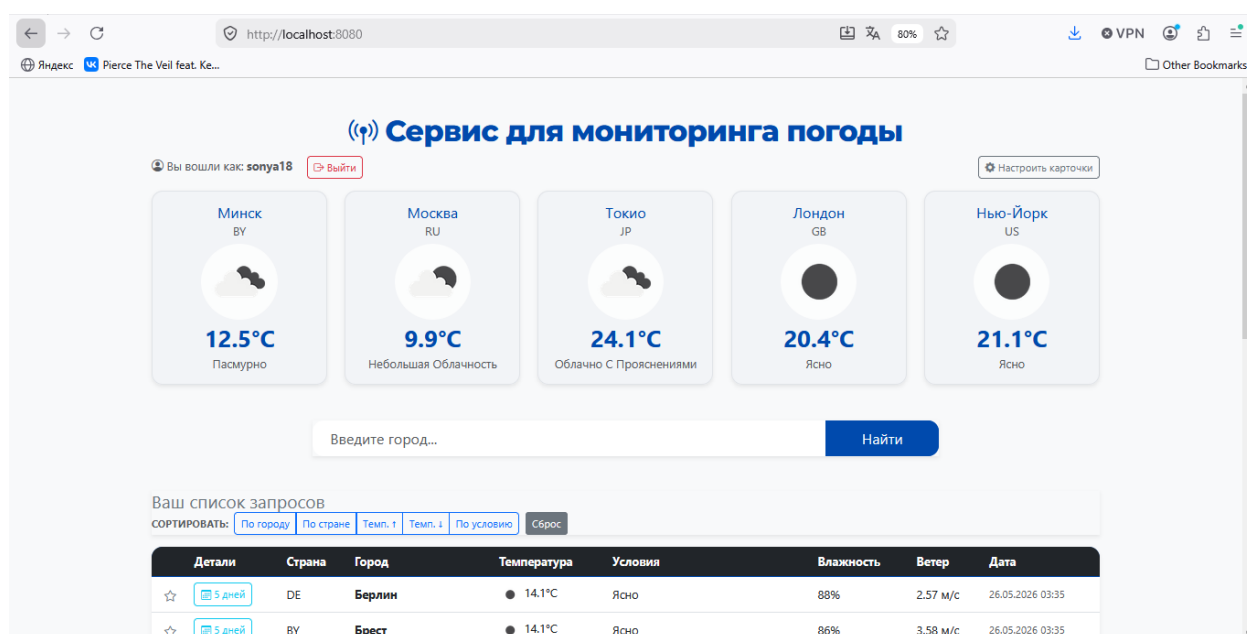


Рисунок 5.1 – Главное окно веб-приложения

Главное окно приложения содержит верхнюю панель управления, блок информационных карточек популярных городов, интерактивную строку поиска и таблицу истории запросов. Интерфейс спроектирован таким образом, чтобы обеспечить быстрый доступ к метеоданным без лишней навигации.

Для получения доступа к персональным функциям системы новому пользователю необходимо пройти процедуру регистрации. Регистрация выполняется через специальную форму, содержащую поля для ввода имени пользователя и пароля.

На рисунке 5.2 представлена форма регистрации пользователя.

A registration form titled "Регистрация" (Registration) is centered on a light gray background. The form is a white rounded rectangle with a thin gray border. It contains two input fields: "Придумайте логин" (Choose a login) and "Придумайте пароль" (Choose a password). Below these fields is a prominent green button with the text "Зарегистрироваться" (Register). At the bottom of the form, there is a link that says "Уже есть аккаунт? [Войти](#)" (Already have an account? [Login](#)).

Рисунок 5.2 – Форма регистрации пользователя

После успешной регистрации пользователь может выполнить вход в систему, указав свои учетные данные. После авторизации в верхней части интерфейса отображается имя текущего пользователя, и становятся доступны функции управления избранными локациями, настройки панели быстрого доступа и ведения истории запросов. На рисунке 5.3 представлена форма авторизации пользователя.

A login form titled "Вход" (Login) is centered on a light gray background. The form is a white rounded rectangle with a thin gray border. It contains two input fields: the first contains the text "sonya18" and the second contains masked characters ".....". Below these fields is a prominent blue button with the text "Войти" (Login). At the bottom of the form, there is a link that says "Создать аккаунт" (Create account).

Рисунок 5.3 – Форма авторизации пользователя

Для поиска актуальной погоды в конкретном населенном пункте используется центральная поисковая строка. В системе реализована функция автодополнения: при вводе названия города приложение асинхронно

запрашивает подходящие варианты у географической службы и отображает их в виде выпадающего списка. На рисунке 5.4 представлен процесс поиска города с использованием подсказок.

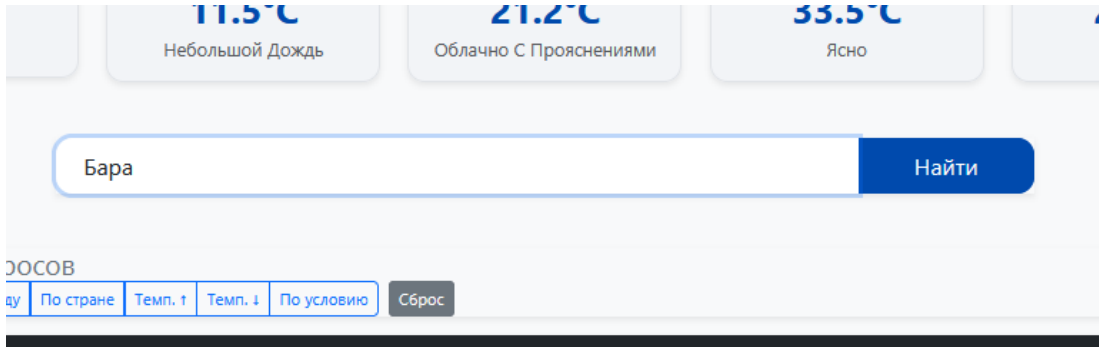


Рисунок 5.4 – Использование функции поиска

После нажатия кнопки «Найти» система выполняет запрос к метеослужбе и добавляет полученные данные в таблицу истории. Для каждого города в таблице отображаются:

- текущая температура (в градусах Цельсия) и графическая иконка состояния атмосферы;
- страна расположения населенного пункта;
- текстовое описание погодных условий;
- физические параметры (влажность воздуха и скорость ветра);
- дата и время получения метеоданных.

Пользователь может добавлять наиболее важные для него города в список «Избранного», нажимая на иконку звезды рядом с названием города. Избранные города автоматически закрепляются в верхней части списка при базовой сортировке. На рисунке 5.5 представлено добавление города в список избранного.

Ваш список запросов

СОРТИРОВАТЬ: [По городу](#) [По стране](#) [Темп. ↑](#) [Темп. ↓](#) [По условию](#) [Сброс](#)

Детали	Страна	Город	Температура	Условия	Влажность	Ветер	Дата
★ 5 дней	BY	Брест	☁ 24.0°C	Переменная Облачность	38%	11.0 м/с	26.05.2026 16:31
★ 5 дней	PL	Варшава	● 26.3°C	Ясно	39%	8.23 м/с	26.05.2026 16:31
★ 5 дней	BY	Гродно	☁ 20.9°C	Облачно С Прояснениями	49%	10.0 м/с	26.05.2026 16:31
☆ 5 дней	BY	Барановичи	☁ 19.2°C	Пасмурно	68%	7.41 м/с	26.05.2026 16:31
☆ 5 дней	DE	Берлин	● 29.3°C	Ясно	41%	4.12 м/с	26.05.2026 16:31
☆ 5 дней	BY	Витебск	● 17.4°C	Ясно	29%	8.0 м/с	26.05.2026 16:31

Рисунок 5.5 – Управление списком избранных локаций

Для детального анализа поисковой истории в приложении реализован блок кнопок мгновенной сортировки. Пользователь может в один клик упорядочить данные в таблице по алфавиту (название города или страна), по значению температуры (возрастание/убывание) или по погодным условиям.

На рисунке 5.6 представлен результат сортировки данных по температуре.

Детали	Страна	Город	Температура	Условия	Влажность	Ветер	Дата
★	BY	Гродно	☁ 20.9°C	Облачно С Прояснениями	49%	10.0 м/с	26.05.2026 16:31
★	BY	Брест	☁ 24.0°C	Переменная Облачность	38%	11.0 м/с	26.05.2026 16:31
★	PL	Варшава	☀ 26.3°C	Ясно	39%	8.23 м/с	26.05.2026 16:31
☆	RU	Москва	☁ 12.6°C	Небольшой Дождь	54%	7.31 м/с	26.05.2026 16:31
☆	RU	Санкт-Петербург	☁ 13.9°C	Переменная Облачность	69%	8.0 м/с	26.05.2026 16:31
☆	BY	Витебск	☀ 17.4°C	Ясно	29%	8.0 м/с	26.05.2026 16:31
☆	BY	Минск	☁ 18.5°C	Пасмурно	58%	6.07 м/с	26.05.2026 16:31
☆	RU	Сочи	☁ 19.1°C	Пасмурно	82%	2.87 м/с	26.05.2026 16:31
☆	BY	Барановичи	☁ 19.2°C	Пасмурно	68%	7.41 м/с	26.05.2026 16:31
☆	UA	Киев	☁ 20.7°C	Пасмурно	57%	5.31 м/с	26.05.2026 16:31
☆	CN	Пекин	☁ 20.9°C	Пасмурно	72%	3.17 м/с	26.05.2026 16:31

Рисунок 5.6 – Сортировка метеоданных в таблице

При необходимости получения подробной метеорологической сводки пользователь может перейти к просмотру пятидневного прогноза, нажав кнопку «5 дней» в соответствующей строке таблицы. На открывшейся странице отображается динамика изменений температуры и погодных условий с детализацией по датам.

На рисунке 5.7 представлен интерфейс прогноза погоды на 5 дней.

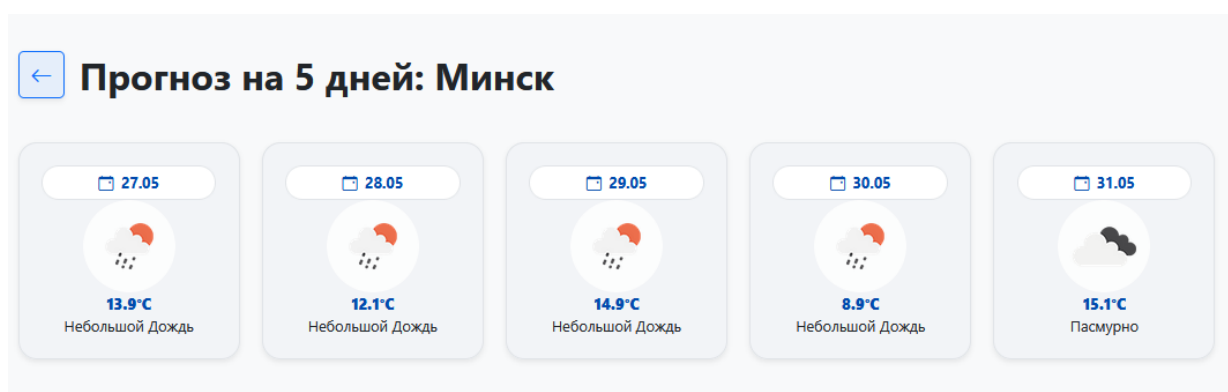


Рисунок 5.7 – Прогноз погоды на 5 дней

Для персонализации главной страницы в приложении реализована функция настройки панели быстрого доступа. Пользователь может вызвать модальное окно настроек и указать пять приоритетных городов, данные по которым будут всегда отображаться в виде крупных информационных карточек в верхней части экрана.

На рисунке 5.8 представлено окно настройки популярных городов.

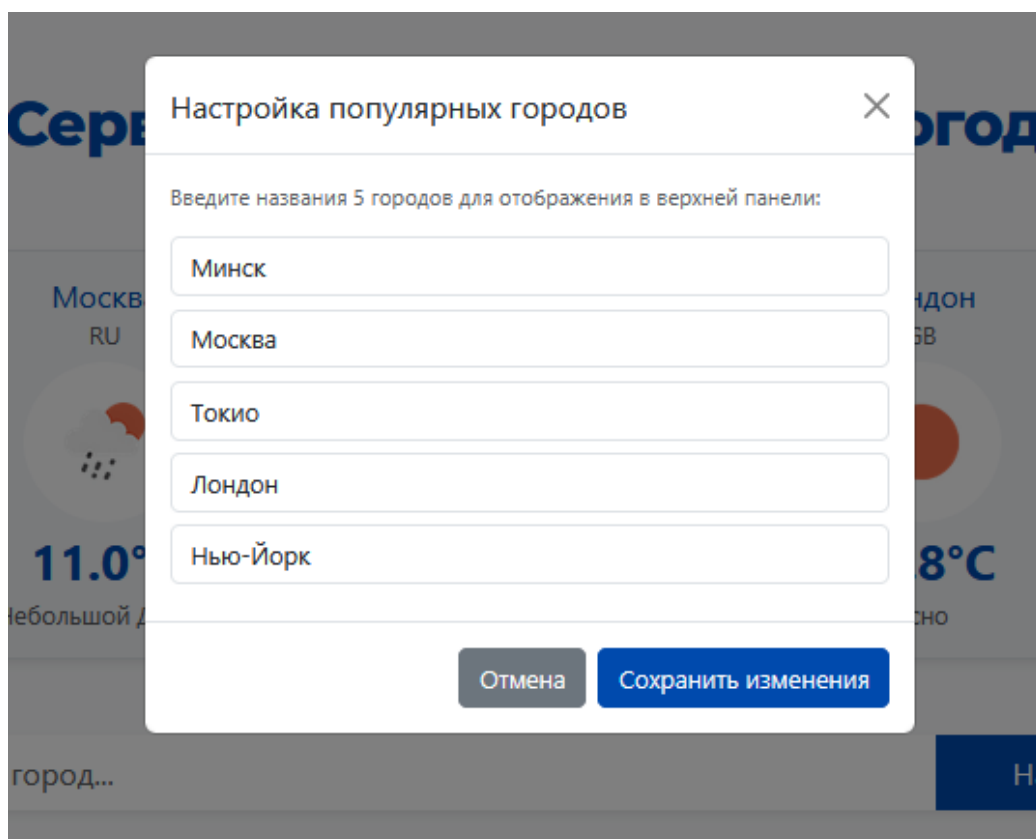


Рисунок 5.8 – Настройка панели быстрого доступа

Таким образом, пользователь системы может выполнять регистрацию и авторизацию, осуществлять поиск метеоданных, формировать персональные списки избранного, просматривать детализированные прогнозы и гибко настраивать интерфейс под собственные нужды. Система обеспечивает централизованное хранение всех настроек в профиле пользователя, гарантируя актуальность данных при каждом входе в приложение.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсового проекта было разработано многопользовательское веб-приложение для персонализированного мониторинга метеорологических данных. Система обеспечивает наглядный интерфейс для взаимодействия с актуальной климатической информацией и поддерживает ключевые функции современных погодных агрегаторов.

В ходе работы были реализованы следующие возможности: поиск городов с автодополнением, просмотр пятидневных прогнозов, формирование списков избранного, а также полная персонализация через систему регистрации. Пользователи получили возможность вести личную историю запросов и настраивать панель быстрого доступа из пяти приоритетных городов.

Серверная часть приложения построена на языке Java с использованием фреймворка Spring Boot и встроенного сервера Tomcat. Безопасность и разграничение прав доступа обеспечиваются модулем Spring Security с хэшированием паролей по алгоритму BCrypt. Особенностью архитектуры является использование локального хранилища на базе JSON-файлов, что гарантирует персистентность данных без необходимости развертывания сторонних СУБД.

Клиентская часть реализована на базе Thymeleaf и Bootstrap с применением JavaScript. Использование асинхронных запросов (Fetch API[16]) и клиентских алгоритмов обработки данных позволило внедрить динамические подсказки и мгновенную сортировку таблиц без полной перезагрузки страниц, что существенно повысило отзывчивость интерфейса.

Ключевой технической особенностью системы является эффективная интеграция с внешним REST API OpenWeatherMap. Реализованный механизм обработки внешних данных обеспечивает надежное получение оперативных сводок, их десериализацию в программные объекты и адаптивную визуализацию.

Перспективы развития проекта включают внедрение интерактивных погодных карт и системы экстренных уведомлений об опасных метеоусловиях. Таким образом, разработанное веб-приложение представляет собой завершенную работоспособную систему, обеспечивающую эффективное взаимодействие клиентской и серверной частей и поддерживающую полный цикл обработки метеоданных в реальном времени.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

- [1] Гисметео — прогноз погоды [Электронный ресурс]. – Режим доступа: <https://www.gismeteo.ru>.
- [2] Яндекс Погода — сервис для мониторинга метеоусловий [Электронный ресурс]. – Режим доступа: <https://yandex.ru/pogoda>.
- [3] OpenWeatherMap: Погодный API для разработчиков [Электронный ресурс]. – Режим доступа: <https://openweathermap.org/api>.
- [4] Что такое REST? // Habr [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/483202/>.
- [5] Spring Boot Reference Documentation [Электронный ресурс]. – Режим доступа: <https://docs.spring.io/spring-boot/docs/current/reference/html/>.
- [6] Thymeleaf Documentation [Электронный ресурс]. – Режим доступа: <https://www.thymeleaf.org/documentation.html>.
- [7] JSON (JavaScript Object Notation): формат обмена данными // Википедия [Электронный ресурс]. – Режим доступа: <https://ru.wikipedia.org/wiki/JSON>.
- [8] Java Documentation [Электронный ресурс]. – Режим доступа: <https://docs.oracle.com/en/java/>.
- [9] Spring Security Reference [Электронный ресурс]. – Режим доступа: <https://docs.spring.io/spring-security/reference/>.
- [10] BCrypt — адаптивная функция хеширования паролей // Википедия [Электронный ресурс]. – Режим доступа: <https://ru.wikipedia.org/wiki/Bcrypt>.
- [11] Основы CSS: каскадные таблицы стилей // MDN Web Docs (на русском) [Электронный ресурс]. – Режим доступа: <https://developer.mozilla.org/ru/docs/Web/CSS>.
- [12] Bootstrap Documentation [Электронный ресурс]. – Режим доступа: <https://getbootstrap.com/docs/>.
- [13] Современный учебник JavaScript // Learn JavaScript [Электронный ресурс]. – Режим доступа: <https://learn.javascript.ru/>.
- [14] Протокол HTTP: основные понятия и структура // Википедия [Электронный ресурс]. – Режим доступа: <https://ru.wikipedia.org/wiki/HTTP>.
- [15] Jackson JSON Processor [Электронный ресурс]. – Режим доступа: <https://github.com/FasterXML/jackson>.
- [16] Fetch API: современный способ отправки сетевых запросов // JavaScript.info [Электронный ресурс]. – Режим доступа: <https://learn.javascript.ru/fetch>.
- [17] HTML5 — язык разметки гипертекстовых документов // MDN Web Docs (на русском) [Электронный ресурс]. – Режим доступа: <https://developer.mozilla.org/ru/docs/Web/HTML>.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода с комментариями

Содержимое файла WeatherController.java:

```
package com.weather.controller;

import com.weather.model.*;
import com.weather.repository.UserRepository;
import com.weather.service.WeatherApiService;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.util.UriUtils;

import java.nio.charset.StandardCharsets;
import java.security.Principal;
import java.util.*;

@Controller
public class WeatherController {

    private final WeatherApiService apiService;
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    // Список городов по умолчанию для заполнения таблицы при первом входе
    private final List<String> tableCities = List.of(
        "Минск", "Брест", "Гродно", "Витебск", "Могилев",
        "Москва", "Санкт-Петербург", "Киев", "Варшава", "Прага",
        "Берлин", "Мадрид", "Астана", "Пекин", "Дубай", "Стамбул",
        "Сеул", "Ереван"
    );

    // Внедрение зависимостей через конструктор (инъекция зависимостей
    Spring)

    public WeatherController(WeatherApiService apiService,
```



```

        UserRepository userRepository,
        PasswordEncoder passwordEncoder) {

    this.apiService = apiService;
    this.userRepository = userRepository;
    this.passwordEncoder = passwordEncoder;
}

// Обработка главной страницы: сбор персональных данных и актуализация
погоды
@GetMapping("/")
public String index(Model model, Principal principal,
                    @RequestParam(required = false) String sort,
                    @RequestParam(required = false) String newCity) {

    // Идентификация текущего пользователя через сессию Spring Security
    User user =
userRepository.findByUsername(principal.getName()).orElseThrow();
    List<WeatherData> userHistory = user.getHistory();
    List<String> cardCities = user.getPopularCities();

    // Формирование данных для верхней панели (5 популярных городов)
    List<WeatherData> popularCards = new ArrayList<>();
    for (String city : cardCities) {
        try {
            WeatherData data = apiService.getRemoteWeather(city);
            if (data != null) {
                // Синхронизация статуса "избранного" из профиля
пользователя
                userHistory.stream()
                    .filter(h ->
h.getCity().equalsIgnoreCase(data.getCity()))
                    .findFirst()
                    .ifPresent(h ->
data.setFavorite(h.isFavorite()));
                popularCards.add(data);
            }
        } catch (Exception ignored) {}
    }
}

```

```

        // Логика построения общей таблицы: объединение истории пользователя
        и списка городов по умолчанию

        List<WeatherData> mainTable = new ArrayList<>();

        Set<String> allCitiesToDisplay = new LinkedHashSet<>();

        userHistory.forEach(h -> allCitiesToDisplay.add(h.getCity()));

        allCitiesToDisplay.addAll(tableCities);

        for (String cityName : allCitiesToDisplay) {
            try {
                WeatherData freshData =
                apiService.getRemoteWeather(cityName);

                if (freshData != null) {
                    userHistory.stream()
                                .filter(h ->
                h.getCity().equalsIgnoreCase(freshData.getCity()))
                                .findFirst()
                                .ifPresent(h ->
                freshData.setFavorite(h.isFavorite()));

                    mainTable.add(freshData);
                }
            } catch (Exception e) {
                // Если API недоступно, отображаем данные из локального кэша
                истории

                userHistory.stream()
                            .filter(h -> h.getCity().equalsIgnoreCase(cityName))
                            .findFirst()
                            .ifPresent(mainTable::add);
            }
        }

        // Сортировка: сначала избранные города, затем по выбору пользователя

        Comparator<WeatherData> comp =
        Comparator.comparing(WeatherData::isFavorite).reversed();

        if (sort != null) {
            switch (sort) {
                case "temp" -> comp =
                comp.thenComparingDouble(WeatherData::getTemperature);
            }
        }
    }
}

```

```

        case "temp_desc" -> comp =
comp.thenComparing(Comparator.comparingDouble(WeatherData::getTemperature).re
versed());

        case "city" -> comp =
comp.thenComparing(WeatherData::getCity, String.CASE_INSENSITIVE_ORDER);

        case "condition" -> comp =
comp.thenComparing(WeatherData::getCondition);

    }

    } else {

        comp = comp.thenComparing(WeatherData::getCity,
String.CASE_INSENSITIVE_ORDER);

    }

    mainTable.sort(comp);

    // Передача данных в шаблонизатор Thymeleaf
    model.addAttribute("popular", popularCards);
    model.addAttribute("history", mainTable);
    model.addAttribute("highlightedCity", newCity != null ? newCity :
"" );

    model.addAttribute("currentPopularNames", cardCities);

    return "index";

}

// Обновление персональных настроек панели быстрого доступа (5 городов)
@PostMapping("/update-popular")

public String updatePopular(@RequestParam("cities") List<String> cities,
Principal principal) {

    User user = userRepository.findByUsername(principal.getName()).get();

    List<String> filtered = cities.stream()

        .filter(c -> c != null && !c.isBlank())

        .limit(5) // Ограничение количества элементов для сохранения
дизайна интерфейса

        .toList();

    user.setPopularCities(filtered);

    userRepository.save(user); // Синхронизация с JSON-хранилищем

    return "redirect:/";

}

```

```

// Обработка поискового запроса: получение данных из API и сохранение в
// историю юзера

@PostMapping("/fetch")
public String fetch(@RequestParam String city, Principal principal) {
    User user = userRepository.findByUsername(principal.getName()).get();
    String encodedCity = "";
    try {
        WeatherData data = apiService.getRemoteWeather(city);
        if (data != null) {
            // Предотвращение дублирования записей в истории запросов
            user.getHistory().removeIf(d ->
d.getCity().equalsIgnoreCase(data.getCity()));
            user.getHistory().add(data);
            userRepository.save(user);
            // Кодирование URL для корректной передачи кириллических имен
городов
            encodedCity = UriUtils.encode(data.getCity(),
StandardCharsets.UTF_8);
        }
    } catch (Exception ignored) {}
    return "redirect:/?newCity=" + encodedCity;
}

// Переключение статуса "Избранное" для конкретной локации
@PostMapping("/favorite")
public String toggleFavorite(@RequestParam String city, Principal
principal) {
    User user = userRepository.findByUsername(principal.getName()).get();
    boolean found = false;
    for (WeatherData d : user.getHistory()) {
        if (d.getCity().equalsIgnoreCase(city)) {
            d.setFavorite(!d.isFavorite());
            found = true;
            break;
        }
    }
    // Если города нет в истории, при отметке "избранное" он
автоматически добавляется из API

```

```

        if (!found) {
            try {
                WeatherData data = apiService.getRemoteWeather(city);
                if (data != null) {
                    data.setFavorite(true);
                    user.getHistory().add(data);
                }
            } catch (Exception ignored) {}
        }
        userRepository.save(user);
        return "redirect:/";
    }

    // Обработка регистрации новой учетной записи
    @PostMapping("/register")
    public String registerUser(@RequestParam String username, @RequestParam
String password) {
        if (userRepository.findByUsername(username).isPresent()) {
            return "redirect:/register?error"; // Проверка на уникальность
логина
        }

        // Хэширование пароля перед сохранением
        User newUser = new User(username, passwordEncoder.encode(password));
        userRepository.save(newUser);
        return "redirect:/login";
    }

    // Запрос и отображение детализированного прогноза на 5 дней
    @GetMapping("/forecast/{city}")
    public String forecast(@PathVariable String city, Model model) {
        try {
            List<ForecastDay> days = apiService.getForecast(city);
            model.addAttribute("city", city);
            model.addAttribute("days", days);
        } catch (Exception e) { return "redirect:/"; }
        return "forecast";
    }
}

```

```
}
```

Содержимое файла WeatherApiService.java:

```
package com.weather.service;

import com.fasterxml.jackson.databind.JsonNode;
import com.weather.model.WeatherData;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import com.weather.model.ForecastDay;
import java.util.ArrayList;
import java.util.List;
import org.springframework.beans.factory.annotation.Value;

@Service
public class WeatherApiService {

    // Внедрение ключа API из файла конфигурации (application.properties)
    @Value("${weather.api.key}")
    private String apiKey;

    // Метод получения краткосрочного прогноза (на 5 дней)
    public List<ForecastDay> getForecast(String city) {
        RestTemplate restTemplate = new RestTemplate();

        // Формирование URL для REST-запроса с указанием метрической системы
        // и русского языка
        String url =
            String.format("http://api.openweathermap.org/data/2.5/forecast?q=%s&appid=%s&units=metric&lang=ru", city, apiKey);

        // Выполнение сетевого запроса и получение ответа в виде дерева JSON
        JsonNode root = restTemplate.getForObject(url, JsonNode.class);
        List<ForecastDay> forecast = new ArrayList<>();
        if (root != null && root.has("list")) {
            // Фильтрация данных: из массива замеров выбираем только
            // ежедневный прогноз на полдень
        }
    }
}
```

```

        for (JsonNode node : root.path("list")) {
            String dtTxt = node.path("dt_txt").asText();
            if (dtTxt.contains("12:00:00")) {
                forecast.add(new ForecastDay(
                    dtTxt.substring(0, 10), // Извлечение даты
                    node.path("main").path("temp").asDouble(),
                    node.path("weather").get(0).path("description").asText(),
                    node.path("weather").get(0).path("icon").asText()
                ));
            }
        }
    }
    return forecast;
}

// Получение актуальной метеорологической сводки для города
public WeatherData getRemoteWeather(String city) {
    RestTemplate restTemplate = new RestTemplate();

    String url =
String.format("http://api.openweathermap.org/data/2.5/weather?q=%s&appid=%s&u
nits=metric&lang=ru", city, apiKey);

    JsonNode root = restTemplate.getForObject(url, JsonNode.class);
    return new WeatherData(
        root.path("name").asText(), // Наименование города
        root.path("sys").path("country").asText(), // Код страны
        root.path("main").path("temp").asDouble(), // Температура
        root.path("weather").get(0).path("description").asText(), //
Текстовое описание
        root.path("main").path("humidity").asInt(), // Влажность
        root.path("wind").path("speed").asDouble(), // Скорость ветра

        LocalDateTime.now().format(DateTimeFormatter.ofPattern("dd.MM.yyyy HH:mm")),
        // Время замера

        root.path("weather").get(0).path("icon").asText() //
Идентификатор графической иконки
    );
}
}

```

ПРИЛОЖЕНИЕ Б
(обязательное)
Схема алгоритма работы системы